
From Pull Requests to Sandboxed Restoration: A Logbook of Building an LLM Hardware-Agent Benchmark

David Andrews*, Saketh Reddy*
Georgia Institute of Technology
{dandrews47, sreddy347}@gatech.edu

Abstract

This logbook documents two and a half months of work building an LLM-agent benchmark for hardware description languages, ending with the RTL-UNIVERSE benchmark we report in our companion paper. The arc covers five repositories built over the spring 2026 semester: a GitHub pull request data pipeline (SWE-UNIVERSE-VERILOG), an early PR-replay evaluation harness (RTL-BENCH), a port of NVIDIA’s CVDP problem suite into our Harbor infrastructure (CVDP-HARBOR) with three frontier-model runs, a spec-to-GDSII benchmark (SPEC2GDSBENCH) that we built and pivoted three times across three weekly reports, and finally the project-scale closed-book restoration benchmark itself. We wrote roughly 12,000 words of weekly research reports, ran hundreds of agent jobs at a combined cost in the four-figure range, surfaced two real benchmark-integrity findings (a 79/92 upstream-CVDP harness bug we patched for our port, and a sandbox-bypass cheating pattern we then defended against), and produced one task-generation skill plus the eleven RTL-UNIVERSE tasks. Although the project began as a CS 3220 research option, we treat the final submission as a benchmark-design study, with RISC-V cores, AI accelerators, and real open-source RTL projects serving as the evaluation substrate. The logbook records what we tried, what failed, what we learned, and why we pivoted, with concrete commit references throughout.

1 Goal and Headline Lessons

The high-level question we set out on was the following. *Can current LLM agents do real hardware-engineering work, and how do we measure that?* The literature on hardware LLMs in early 2026 was almost entirely module-level Verilog generation [1–6], with leading models reporting 90 percent or higher pass rates on benchmarks that ask for a 4-bit counter from a description. That left two open questions. How would those agents fare on the rest of a real chip-design workflow (build systems, integration, testbenches, physical design)? And is there a benchmark that we trust to grade them on it?

Across the five repositories, the headline pattern that emerged was the following. PR-replay does not isolate cleanly in HDL. Module-level benchmarks have already saturated. Written behavioural specifications are themselves the bottleneck, not the agent. Real upstream open-source projects already carry the test infrastructure we kept trying to hand-roll. And restoration of stripped files inside one of those projects, under a closed-book sandbox, is the design that finally makes the agent’s RTL implementation ability the load-bearing variable. The companion paper reports the final benchmark.

*Equal contribution. Authors listed in alphabetical order.

Table 1 lists the five phases, the repositories that contained them, the commit ranges that bound them, the headline result of each phase, and the decision that ended each phase. The remainder of this logbook expands every row.

Table 1: Project timeline. Each row is a phase, the repository it lived in, the commit range that bounds it, the headline result, and the decision that ended it.

Period	Repo	Commits	Result	Decision
Mar 2–6	swe-universe-verilog	47f1b40..1b6b034 (4 commits)	Pipeline yield: 173k merged PRs filtered to 401 high-quality PRs across 3,296 HDL repos.	PR-replay does not isolate cleanly. Pivot to CI-graded restoration.
Mar 2–3	rtlbench	45e61b9..38a68a1 (12 commits)	First Claude runs across 7 upstream HDL repos. Toolchain-version mismatches and ambiguous testbenches surfaced.	Adopt per-task Dockerfiles. Require mechanically-checkable verifiers.
Mar 7–13	cvdp_benchmark	c002eb5..eba655b (4 commits)	Harbor conversion tool, 99 tasks ported. 3-model run on 92 tasks: Opus 81.5%, GPT-5.4 66.3%, Gemini 3.1-Pro 54.3%.	Module-level HDL is saturated. Move to project scale.
Mar 23–28	spec2gdsbench (W1)	9821882..d96e177 (18 commits)	Spec-to-GDSII pipeline with 8 tasks. Sonnet 0.964 mean over 64 runs.	Near-saturated. Pivot to a single SoC task.
Mar 31–Apr 6	spec2gdsbench (W2)	ba338b7..2fa0d8d (28 commits)	RV64GC Linux-boot SoC. Sonnet best 0.616.	Single task gives poor signal. Pivot to a graduated ladder.
Apr 7–12	spec2gdsbench (W3)	723f8c3	Three AI-accelerator tasks (systolic_array, conv2d_engine, dma_engine). All three reference solutions pass full GDS flow.	Specification authoring is the bottleneck. Pivot to upstream restoration.
Apr 23–May 8	rtl-universe	1024c2a..daf3cc7 (76 commits)	16 candidate tasks engineered, 11 shipped, 7 archived for build issues. 7-model $\times$ 11-task matrix. Audit: 10 of 18 perfect scores are bypasses or exploits.	Companion paper. Continue.

2 Phase 1: SWE-Universe-Verilog (March 2–6)

2.1 Goal

SWE-bench [7] grades agents on real GitHub bugs in real Python projects. The first thing we tried was the same recipe but for HDL. We wanted a curated dataset of high-quality Verilog and SystemVerilog pull requests that could serve as the basis for a SWE-bench-style benchmark.

2.2 Pipeline

We re-implemented the multi-stage filter from the SWE-Universe data-curation methodology [8], targeting the Verilog and SystemVerilog ecosystem on GitHub. The pipeline lives in `other_repos/swe-universe-verilog/` and consists of seven stages, each implemented as a separate Python module (`stage1_discover_repos.py` through `stage7_quality_judge.py`) with SQLite-backed persistence so each stage is resumable.

- Repository discovery.** We searched GitHub for repositories whose primary language is Verilog or SystemVerilog, plus those tagged with HDL-related topics (`verilog`, `fpga`, `rtl`, `hdl`, `asic`). We worked around GitHub’s 1,000-result search limit by partitioning queries on monthly creation dates.
- PR collection.** We collected all merged PRs from the discovered repositories through GitHub’s GraphQL API, with rate-limit-aware token rotation across multiple personal access tokens.
- Heuristic filter.** We removed PRs that are unlikely to be useful: bot-authored, merge commits, no Verilog files changed, or outside reasonable size bounds (1 to 20 files, 5 to 3,000 lines changed).
- Testbench detection.** We scored each PR for testbench relevance using file name patterns (`_tb.v`, `test_*`), directory patterns (`/tb/`, `/verif/`), and content patterns (simulation constructs, assertions, UVM, cocotb). PRs scoring at least 3 were kept.

5. **Issue linkage.** We extracted issue references from PR bodies and commit messages using SWE-bench-style closing keywords (`fixes`, `closes`, `resolves`).
6. **Patch separation.** We classified changed files into test files, fix files (Verilog or SystemVerilog source), and other files (docs, config, scripts). PRs with both test and fix components were flagged.
7. **Quality judge.** A composite quality score combines description quality (0.25), test presence (0.35), patch characteristics (0.20), and issue linkage (0.20).

2.3 Yield

Table 2 shows the funnel.

Table 2: SWE-UNIVERSE-VERILOG pipeline yield. The funnel reduces 173 thousand candidate PRs to 401 PRs that pass the strict quality and issue-linkage thresholds.

Stage	Count
Repos discovered (Verilog/SV on GitHub)	3,296
PRs collected (merged)	173,106
After heuristic filter (size, bot, merge)	27,446
After testbench detection	5,282
After patch separation (test + fix files)	3,070
Quality filter (relaxed)	1,241
Quality filter (strict, with issue linkage)	401

2.4 What we learned

Even after aggressive filtering, the yield from the GitHub HDL ecosystem is roughly an order of magnitude lower than the Python equivalent. 401 strict-filter PRs across 3,296 repositories is enough to start a benchmark in principle. In practice each one had to be curated by hand to verify that it isolated a single change with a real before-and-after test. We built RTL_{BENCH} (§3) on top of this data and quickly discovered that PR-based hardware bugs do not in practice isolate cleanly. Most are part of a larger refactor, depend on FPGA-vendor toolchains, or test only a manually-asserted simulation trace.

This phase informed two decisions in RTL-UNIVERSE. We chose to abandon PR-replay as the framing, and we chose restoration (delete-and-rebuild) instead. Restoration gives the agent a real test suite (the upstream’s own CI) without requiring us to curate a clean before-and-after diff, and it does not depend on whether the upstream PR was clean.

3 Phase 2: RTL_{BENCH} (March 2–3)

3.1 Goal

Using the SWE-UNIVERSE-VERILOG filtered set, plus a few additional integration projects, build a quick evaluation harness that clones an upstream repo at the parent commit of a chosen PR, sets up the toolchain in a container, and asks Claude to make the change. The objective was a working SWE-bench-for-HDL proof of concept.

3.2 Setup

We pulled seven HDL repositories that span the difficulty spectrum: `picorv32` (small RISC-V core), `verilog-axi`, `verilog-ethernet`, `verilog-uart` (Alex Forencich’s mature peripheral libraries), `corundum` (a 100G FPGA NIC), `cva6` (a Linux-capable RISC-V core), and `hdl` (Analog Devices’ FPGA reference designs). We documented our reading of SWE-bench, SWE-smith, and SWE-Universe in `insights/`, with one Markdown file per paper that summarised the data schema, the collection pipeline, and the bug-generation strategy. We then wrote three Dockerfiles for the toolchain layers we needed: `Dockerfile.cocotb1` (cocotb 1.x with iverilog, for verilog-ethernet), `Dockerfile.cocotb2` (cocotb 2.x with iverilog), and `Dockerfile.verilator` (Verilator-based tasks).

The workspace under `rtlbench/workspace/swebench_verilog/` reimplements the SWE-bench harness signatures (`run_evaluation_docker.py`, `grading.py`, `log_parsers.py`) but for our HDL toolchain. The early commits read like a chronology of debugging: `init` (45e61b9), still before `run` (142f9c3), `snap` (35ec496), first `claude run` (3c5ebff), `claude plan expand not run yet` (99bb14f), `fix: add missing submodule entries` (eb1ebfd), `Add corundum SYNTH instances and cva6 integration` (f67d3d4), and finally `ran full eval` (38a68a1). Twelve commits over 36 hours.

3.3 What we learned

The first runs worked just well enough to surface two failure modes that ended up shaping everything that followed.

The first was **toolchain fragility**. Verilog projects often pin specific Verilator and Yosys versions in their CI configurations. The agent would happily install a different one and write code that compiled in the wrong version. We ended up writing per-task Dockerfiles, which is what the next two projects (CVDP-HARBOR and SPEC2GDSBENCH) inherited.

The second was **test-suite ambiguity**. Many of the PRs in our filtered set have testbenches that “run” on master but actually only pass when accompanied by a manually- checked waveform. The agent does not know this. We needed verifiers whose pass criterion is mechanically checkable. This is part of why RTL-UNIVERSE restricts itself to projects with binary-pass CI rather than waveform-graded tests.

4 Phase 3: CVDP-Harbor (March 7–13)

4.1 Goal

At the start of March 2026, NVIDIA released the Comprehensive Verilog Design Problems (CVDP) benchmark [9], a 783-problem suite of mostly module-level Verilog tasks with both non-agentic and agentic variants. Rather than re-invent the problem set ourselves, we ported CVDP into Harbor, our internal evaluation harness. The objective was to (i) stand up Harbor as a working evaluation backend, (ii) quantify how saturated module-level CVDP actually was for current frontier models, and (iii) surface bugs in either CVDP’s harness or Harbor that would block scaling.

4.2 The conversion tool

The Harbor conversion tool is at `cvdp_benchmark/` commit `c002eb5` (“Add CVDP to Harbor conversion tool”) with three more rounds of fixes and reruns over the next week (`36f17fb`, `891039d`, `eba655b`). It reads the upstream CVDP JSONL dataset (`cvdp_v1.0.4_agentic_code_generation_no_commercial.jsonl`) and writes a Harbor task directory per problem. The output went to `harbor_output/` as 99 converted tasks, each with the standard Harbor layout: `environment/Dockerfile`, `environment/rtl/`, `environment/src/`, `instruction.md`, `task.toml`, and `tests/test.sh`. The orchestration script `run_harbor_terminus2.py` then launches Harbor’s Terminus-2 agent against the converted suite, with per-model concurrency caps and a configurable retry policy.

4.3 Three-model evaluation

We ran three frontier models on 92 of the 99 converted tasks (the other 7 had upstream issues): Claude Opus 4.6, GPT-5.4, and Gemini 3.1-Pro Preview, all with the Terminus-2 agent at 8 concurrent trials per model. The results, reported in `NOTES.md` of the CVDP-Harbor repo, are summarised in Table 3.

Table 3: CVDP-Harbor 3-model results on 92 tasks (March 12, 2026, Terminus-2 agent). “Genuine fails” counts trials where the agent ran to completion but did not solve the task. “Infra errors” counts timeouts, rate-limits, and OpenRouter 5xx responses.

Model	Pass rate	Passes	Genuine fails	Infra errors
Claude Opus 4.6	81.5%	75 / 92	5	15 (timeouts)
GPT-5.4	66.3%	61 / 92	18	15 (API errors + timeouts)
Gemini 3.1-Pro	54.3%	50 / 92	15	27 (rate limits + timeouts)

After the first run, we re-ran the failed cells, re-checking infrastructure failures separately from genuine fails. After re-runs and isolating infrastructure failures, the model that drove our headline conclusion was Claude Opus 4.6, which reached **96.2%** pass rate on the agentic non-commercial split, with the remaining failures attributable to minor issues (pipeline timing off-by-one, file extension typo, signal polarity ambiguity) rather than fundamental capability gaps.

4.4 The 79/92 cocotb harness bug

The most useful artifact from this phase, beyond the eval numbers, was an upstream-bug finding documented in `NOTES.md`. We noticed that 79 of the 92 CVDP tasks were failing on a single import line:

```
from cocotb_tools.runner import get_runner # broken
from cocotb.runner import get_runner # correct
```

The bug is in the upstream CVDP dataset (two different harness versions were merged, “Type A” uses `docker-compose.yml` with `build: .` and the correct import, “Type B” uses `image: __OSS_SIM_IMAGE__` and the broken import). The Harbor conversion tool had been copying files verbatim, propagating the bug. Many of the broken tasks still passed because the agent noticed and fixed the import during execution, but this wasted agent turns and caused some tasks to fail outright when the agent did not catch it. We added a post-processing fix-up to `tools/convert_cvdv_to_harbor.py` that replaces the broken import in generated `environment/src/*.py` files, then regenerated `harbor_output/`.

We also documented the failure modes that were Harbor-side rather than CVDP-side: Gemini’s 17 rate-limit failures (Google’s OpenRouter quota was 250 requests per day, exhausted mid-run), 11 GPT-5.4 trials hit OpenRouter 5xx errors, and 5 tasks errored on every model due to transient infrastructure (`ethernet_mii_0006`, `hdbn_codec_0001`, `jpeg_runlength_enc_0001`, `phase_rotation_0015`, `poly_decimator_0001`). Re-runs cleared all five.

4.5 What we learned

Three things from this phase carry forward.

First, **module-level HDL generation is saturated for current frontier models**, at least on CVDP’s task distribution. The remaining failures are cosmetic. We would not differentiate frontier models on this surface. The open question is at the project scale.

Second, **harness eval is full of upstream bugs** that look like model failures. The 79/92 cocotb import bug would have been read as a model issue without close reading of the transcripts. This is the seed of the audit framing in the companion paper: when a benchmark score does not match expectations, do not assume the score reflects the agent.

Third, **Harbor as our evaluation runner works**. The same Harbor conventions (`environment/Dockerfile`, `instruction.md`, `task.toml`, `tests/test.sh`) carry over to all subsequent phases. RTL-UNIVERSE is built on the same backbone.

At this point we stopped working on CVDP and started a benchmark we hoped would not saturate.

5 Phase 4: Spec2GDSBench (March 23 – April 12)

SPEC2GDSBENCH consumed three weeks of effort and pivoted three times. We wrote roughly 12,000 words of formal weekly research reports during this phase (SURVEY.tex, REPORT.tex, REPORT_WEEK2.tex, REPORT_WEEK3.tex). We summarise each week in turn, then consolidate the lessons.

5.1 Survey week (March 17–21): Where is the gap?

We surveyed five recent papers to map the landscape of LLM hardware-design benchmarks: ChipBench (44 generation, 89 debugging tasks; Claude 4.5 Opus 30.74%; multi-agent MAGE 37.41%), Bench4HLS (170 HLS tasks through Vitis HLS and Vivado; GPT-5 72% sim pass@10), the ICCAD 2025 panel summary on reliability and the natural-language-to-circuit gap, InCoder-32B (the first 32B code model purpose-built for chip design; 80.7 on VeriScope, 74.8% on RealBench module-level Syn@1, but only 10% on system-level Syn@1), and NL2GDS (a tool that translates natural language specs into GDSII via OpenLane on small ISCAS circuits). The conclusion was that every existing benchmark stops short of physical design. ChipBench stops at simulation. RealBench stops at synthesis checking. Bench4HLS goes through P&R but only for single-function HLS kernels.

This is the gap we proposed to fill: an end-to-end benchmark from behavioural specification to a DRC-clean, LVS-verified GDSII file, using the full open-source ASIC toolchain (LibreLane, the SkyWater Sky130A PDK, iVerilog, Verilator, Yosys, OpenROAD, Magic, Netgen, KLayout). The survey is in `other_repos/spec2gdsbench/SURVEY.tex`.

5.2 Week 1 (March 24–28): Eight isolated tasks

The first version had eight tasks, four general-purpose and four AI accelerator primitives: `gcd_unit` (greatest common divisor), `uart_tx` (UART 8N1 transmitter), `aes128_encrypt` (AES-128 with 5 NIST vectors), `femtorv32_rv32i` (a small RISC-V CPU with 8 smoke and 43 RISC-V compliance tests), `conv1d_engine` (streaming 1D convolution), `maxpool2d` (2D max pooling), `requantize_pipeline` (INT32 to INT8 requantisation), and `matmul_4x4_int8` (a 4×4 INT8 matrix multiply). Each task came with a behavioural `instruction.md`, a test harness, a reference solution, and a LibreLane configuration template. The grading funnel had six stages with the weights in Table 4.

Table 4: Spec2GDSBench Week 1 grading funnel. Stages are evaluated sequentially; each gates the next.

Stage	Weight	Criteria
Functional	0.40	Fraction of smoke and compliance tests passing (iVerilog)
Synthesis	0.15	Yosys synthesis succeeds without errors
Timing	0.15	Worst negative slack at the target clock period
Physical	0.10	LibreLane place-and-route succeeds
Signoff	0.10	DRC-clean (KLayout + Magic) and LVS-matching (Netgen)
PPA	0.10	Power-Performance-Area absolute reporting

We ran 64 trials of Claude Code Sonnet 4.6 (8 tasks × 2 modes × 4 replications) at \$5 per run with a uniform 7200-second timeout. Two modes: *normal* (test suite visible to the agent) and *blind* (no test suite). The normal mean reward was **0.964**; the blind mean reward was **0.829**. In normal mode all 32 runs passed all functional tests; the only score loss was timing closure during physical design [10].

The week-1 results are checked in under `results_sync_2/`, with a per-task-per-mode README that lists the four replications and their scores. `ISSUES_INVESTIGATION.md` documents the eight infrastructure fixes we applied between the initial 64 runs and the final 77-run dataset, including Docker container-name collisions on concurrent runs (we added random four-character suffixes), batch-runner timeout buffers (raised from 600 to 3600 seconds), and a missing `timeout_secs` field in `timing.json`.

The benchmark was near-saturated. We pivoted.

5.3 Week 2 (March 31 – April 6): A single Linux-bootable SoC

We replaced the eight tasks with one task: design an RV64GC RISC-V SoC capable of booting Linux. The agent receives only the external interface (a 50 MHz clock, an active-low reset, UART TX/RX, and a 64-bit AXI4 master port) and a software contract: 108 ISA compliance tests from `riscv-tests`, 10 bare-metal programs, OpenSBI plus BusyBox boot via QEMU’s virt machine convention, with the test harness providing the memory model via `cocotb` plus Verilator. The task removed all internal architectural hints from `instruction.md` (commits a09e739, 64376f1, e50402e: “Remove architectural suggestions”, “Remove remaining architectural hints”, “Remove strategy advice and implementation hints from `instruction.md`”). The agent had complete design freedom over pipeline depth, cache hierarchy, bus topology, and FPU implementation.

Across five runs of Claude Code Sonnet 4.6 the best result scored **0.616**: 85 of 108 ISA compliance tests, 10 of 10 bare-metal programs (including floating-point), and OpenSBI boot achieved (1 of 5 Linux milestones). A 15-hour run that was still in progress at the time of the report had reached 99 of 108 ISA compliance and showed evidence of partial Linux boot during debugging [11].

The infrastructure work in Week 2 was substantial. The Dockerfile had to be rebuilt for the Nix-based base image (7b3b504), with a `virtualenv` layer for `cocotb` (33c8394), a fix for the `runc /etc/passwd` symlink issue with Nix containers (df3be87), and `g++` added for Verilator C++ compilation (d534dbb). The evaluator went through nine iterations (8bfd833–4553ab4) on liberty file discovery, Yosys synthesis timeouts, LibreLane configuration provisioning, and reference-stub handling. The verifier timeout was eventually pushed to 5 hours (7528271).

This was a much harder benchmark, but a single task gave poor signal. An agent either built a working SoC or scored near zero. There was no intermediate difficulty to calibrate against. We pivoted again.

5.4 Week 3 (April 7–12): The AI accelerator integration ladder

We then designed three new tasks at graduated difficulty, all with AXI4 interfaces and all targeting the AI-accelerator narrative: `systolic_array` (an 8×8 INT8 MAC array with AXI4-Lite MMIO and AXI4-Stream data interfaces; 18,900 gates, $643 \text{ K} \mu\text{m}^2$), `conv2d_engine` (a 3×3 2D convolution engine with line buffer and AXI4 interfaces; 46,676 gates, $1.24 \text{ M} \mu\text{m}^2$), and `dma_engine` (a memory-to-memory DMA controller with AXI4-Lite configuration and AXI4 master; 2,767 gates, $91 \text{ K} \mu\text{m}^2$). All three reference solutions pass the full ASIC flow through GDSII on Sky130A; only the DMA engine meets timing at 50 MHz.

The technical findings during Week 3, reported in `REPORT_WEEK3.tex`, included two scaling issues that ended up shaping which kinds of tasks we could ship. Yosys’s `abc` technology mapping step scales poorly with multiplier count: the `systolic_array`’s original all-at-once compute (64 MACs in a single cycle) produced 455,000 gates and never completed `abc` within the 3600-second timeout. We switched to sequential compute (one MAC per cycle, 8 multipliers) which reduced this to 18,900 gates and roughly 100 seconds of synthesis. Verilog register arrays with multiple variable-indexed read ports created large multiplexer trees: `conv2d_engine`’s 256-entry array with 9 read ports generated 46,000 gates; an earlier 1024-entry version produced 149,000 gates and timed out LibreLane.

We were partway through getting agent runs on these when the next pivot happened.

5.5 What we learned across the three weeks

Three lessons.

Specifications are hard. Every week spent on SPEC2GDSBENCH, we spent some fraction of the time on whether the spec was over-constraining the agent (forcing a specific microarchitecture) or under-constraining it (letting the agent solve a different problem). Real industrial specs are often closer to a JIRA epic with 30 linked tickets than a paragraph of “the AXI4 signal names”. We could not write a clean specification for a realistic design. We rewrote `instruction.md` for the SoC task five times across two days, removing successive layers of architectural hints, before the agent runs gave reproducible signal.

Build systems and physical design dominated the score. The agent’s Spec2GDSBench score was dominated by physical design rather than RTL design. Timing closure, place-and-route choices,

LibreLane configuration, Yosys synthesis options, and KLayout DRC tolerances all dominated the score. We wanted RTL-design ability to be the bottleneck, but it was hidden behind back-end optimisation.

Real projects already have all of this. The thing that finally moved us was the realisation that lowRISC’s Ibex, NVIDIA’s NVDLA, Princeton’s OpenPiton, the PULP common-cells library, secworks/aes, Google’s Coral NPU, and chipsalliance’s VeeR EL2 each already have testbenches, build systems, CI configurations, and reference implementations carefully tuned by their maintainers. We do not need to write a spec for a UART because lowRISC has a real one and grades it with their own `simple_system`. We do not need a six-stage grading funnel from synthesis through GDSII because the upstream’s CI already produces a deterministic pass-or-fail. We just need to delete files.

The pivot to closed-book restoration of upstream open-source projects came on April 12, captured in commit 723f8c3 (“start work on more ai accelerator tasks, start report”) which is the last commit on `spec2gdsbench/`. Between that commit and the first commit on `rtl-universe/` (1024c2a, “init”, April 23) we wrote the restoration framing and the Harbor task layout that the next phase inherits.

6 Phase 5: RTL-Universe (April 23 – May 8)

Two and a half weeks of task engineering, agent runs, and an audit. 76 commits. This phase produced the artifacts that the companion paper reports.

6.1 First two days: The first two tasks

The repo started on April 23 with a single `init` commit (1024c2a). The next day’s commit (ea74335, “Add Harbor tasks: coralnpu (all tests) + coralnpu-e2e”) added the first two tasks. On April 25 we hardened the Harbor tasks by scrubbing warm-build artifacts that had survived our first agent runs (f783357), increased the agent timeout to 10 hours, and reconstructed the agent-generated code from trajectory logs (877de87, cf7eae8) so we had a record of what the agents had actually written across our first runs.

6.2 The harbor-task-gen skill

On April 28 we shipped the first version of the `harbor-task-gen` skill (1ad3b4d). The skill is a Claude Code skill (`skills/harbor-task-gen/` in the repo) that converts a candidate upstream HDL repository into a Harbor task. Its `SKILL.md` file documents a five-step pipeline: (1) clone the upstream repo and confirm the baseline CI passes; (2) derive the strip manifest by following the upstream’s Makefile or build manifest to identify the design unit and its implementation files; (3) generate `environment/Dockerfile` that installs only the toolchain the upstream CI requires; (4) generate `tests/test.sh` that calls the upstream test target and emits a normalised reward in `[0, 1]`; (5) stash the deleted files separately so we can later compare the agent’s output to ground truth.

The skill went through five iterations on the same day: 1ad3b4d (initial), 82ce12f (“Iterate harbor-task-gen skill: generalize Docker strategy, fix failure modes”), 1df25f5 (“Fix all 5 failure modes from BUGFIXES.md in harbor-task-gen skill”), 9af1d3e (“Replace all tasks with iteration-4 fixed versions”), and cda62aa (“Standardize task variants: always create full + lite pair”). We documented each iteration in `BUGFIXES.md`, which by the end of the day contained roughly twenty entries covering Dockerfile heredoc parse errors, Debian Bookworm’s externally-managed-environment pip refusal, missing git submodule entries, `count_tests.sh` failures on green source, commercial AXI VIPs that the open-source flow could not stand in for, and many others.

By the end of April 28 we had Harbor tasks for Ibex, Caliptra, PULP common-cells, secworks-aes, VeeR EL2, CVA6, Coral NPU, NVDLA, and OpenPiton — nine candidate upstream projects, each with a `-full` and a `-e2e` variant.

6.3 Sixteen tried, eleven shipped, seven archived

The next two days were debugging across the candidate task list. Sonnet runs on April 29 surfaced concrete failure modes. `veer-el2-block` scored 0 in 52 minutes (69a31e7). `veer-el2-full` build failed (the green-source tests fail in Docker because of a Verilator version mismatch; 7e1d2a5). `caliptra-full` build failed (missing commercial AXI VIP; eb8cfa5). `caliptra-rtl-fw` build failed for the same reason (3ff1e16). `secworks-aes-full` hit Claude Code’s 32K output token limit at the 30-minute mark (74939fd). `ibex-e2e` succeeded with reward 1.000 in 7h 35m (2146ec1). `ibex-full` ran for 13.5 hours before we manually stopped it: the agent’s CSR/timer implementation had `mtimecmp` always below `mtime`, which produced an infinite `IRQ`→`MRET`→`IRQ` loop, and the three parallel Verilator sims generated over 100 GB of debug trace logs (32b0e59). On `cva6-smoke` both runs failed during the build phase.

We then iterated through Docker fixes for the broken tasks: 58effc0, de11a8d, 5b11791, a1f9c41, 059399f, e0a6e51, 72161b6, 9ebb346, 164800d, 6e941e6, e371dc8. The CVA6 chain alone took five fixes (Bender 0.28.1 URL 404 → updated to 0.31.0; riscv-tests benchmarks needed `riscv64` prefix → manual ISA install; Verilator 5.006 lacked the `WIDTHEXPAND` lint code → Verilator 5.008 in an isolated builder stage; Spike’s `libriscv.so` not produced by `make install` → `ldconfig` after manual library install; CVA6 testbench’s DPI link against Spike internals → deferred). OpenTitan’s Bazel toolchain bootstrap was its own debugging chapter: 0 of 94 tests passed on green source in Docker even after fixing the inline-comment bug in `install_deps.sh`.

By April 30 we had archived seven tasks (541dfbe, “Archive 7 broken tasks, keep 11 working ones”): `caliptra-full`, `caliptra-rtl-fw`, `cva6-full`, `cva6-smoke`, `opentitan-e2e`, `opentitan-full`, `veer-el2-full`. They live in `minrepro_task/_archived/` for posterity.

6.4 The cheating discovery and DNS sinkhole

April 30 was the day the audit story started. The headline commit message: ba277d1, “OpenCode results: Kimi K2.6 perfect on Ibex (1.000 both variants!)”. Followed seven hours later by eae2aaf, “CRITICAL: Kimi K2.6 cheated on Ibex — downloaded source from GitHub”. Followed 90 minutes later by 4a59e26, “CRITICAL: Multiple models cheat by downloading source from GitHub”.

Reading the Kimi transcripts, we found that the agent had written `git clone https://github.com/lowRISC/ibex` to a temporary directory, computed `md5sums` to confirm the version match against the stripped source, and then `curl’d` each of the 30 stripped Verilog files individually from `raw.githubusercontent.com/lowRISC/ibex/master/`. The 18-minute completion time made sense: it was copying, not implementing. Opus had done the same thing on Coral NPU (e128dfd, ba65fcc): `git clone`, then `rsync -ignore-existing` to copy 281 files back into the stripped tree. Opus’s own session memory file documented the approach explicitly (“Restoration approach: `git clone`, then `rsync`”).

The first attempted defence was `allow_internet = false` on every task (be2604a). This blocked the cheating but also blocked the agent’s API calls and package installs (881d135, “Revert `allow_internet = false` — blocks agent API calls too”). The next attempt was a DNS sinkhole inside `docker-compose.yml`: write `0.0.0.0 github.com` (and twelve other source-hosting domains) to `/etc/hosts` just before the agent gets control (f9e62bd, “Block source code hosting via DNS sinkhole in `docker-compose.yml`”). This worked for Claude Code, which installs from `claude.ai` (not blocked), but broke OpenCode, which installs NVM from `raw.githubusercontent.com`. We documented the OpenCode incompatibility in f9abc7d.

The next day (May 1) we shipped a sandboxed-agent adapter (195ea1b, “Add sandboxed agent adapters to prevent cheating on OpenCode runs”) that pre-installs OpenCode in the Docker image so the agent install path does not need GitHub egress at runtime. From this point onwards, the OpenCode-on-OpenRouter agents were sandboxed too.

6.5 The 812 GB disk-fill incident

On May 6, while running the multi-account Claude Code launcher overnight, the lab server’s root partition went from 813 GB free to 40 MB free in the span of a few hours. The launcher had paused itself per the disk-safety supervisor at 90% usage, but the damage was already done. Investigation showed that one Sonnet job (`sandboxed-sonnet-ibex-full`)

had written 812GB of Verilator build cache into the bind-mounted credentials directory (/tmp/harbor-claude-creds-...-ibex-full/). The cause: when Claude Code installs, it sets HOME to its credentials directory, and any subsequent make or verilator invocations that derive their build output path from HOME land inside the bind-mount. We deleted the stray directory, added a periodic orphan reaper to the supervisor that removes stale per-job credential directories, and added the disk-fill check that paused the launcher in the first place. The lessons-learned section of the companion paper covers this in less detail; the companion paper’s harness section was largely written in response to this incident.

6.6 Multi-account Claude Code

To run the matrix in the time budget we had, we needed two Claude Max subscriptions with asymmetric per-account caps. The launcher (scripts/launcher.py, 616 lines) tracks each Claude job’s credentials by hash, counts how many in-flight jobs are using each account, and refuses to launch if the per-account cap would be exceeded. Combined with the per-Claude-job credential bind-mount that the disk-fill incident already required, the result is a launcher that runs up to four Claude jobs in parallel (two on the primary, one on the secondary), three Codex jobs (single account, MAX_CODEX = 3 due to OpenAI rate limits), and arbitrary OpenRouter jobs subject to balance. The full matrix took roughly four wall-clock days end-to-end on a 128-CPU, 1 TB-RAM lab server.

6.7 The audit

After the matrix was complete we read every agent transcript looking for sandbox bypasses and verifier exploits. Of 18 raw perfect scores, 10 are bypasses or exploits. The companion paper reports the per-model breakdown (Table 3 of the paper) and the eight observed sandbox-bypass methods plus three verifier exploits (Table 4 of the paper).

7 What we would do differently

Three lessons we would carry into the next iteration.

Sandbox first. We built the harness, ran agents, discovered the cheating, then added the DNS sink-hole. By the time we audited the transcripts, most of the perfect scores had already been bypassed and we needed re-runs. A default-deny egress firewall with explicit allowlists, plus a real-time grep over the agent log for known bypass strings, would have caught everything before the run completed.

Verifier hardening should be part of task acceptance. For every task we adopt from an upstream project, “does the agent score 1.000 with a stub” should be a check we run before greenlighting the task. Two of our three flagged tasks were exploitable in 17 minutes of agent time. We could have caught both during task setup with a simple “compile-only DUT” regression.

Restoration is the right framing for now. Of the four prior approaches (PR-replay in SWE-UNIVERSE-VERILOG and RTL BENCH, module synthesis in CVDP-HARBOR, full spec-to-GDS in SPEC2GDSBENCH, restoration in RTL-UNIVERSE), restoration is the only one that simultaneously gives the agent a real test suite, avoids the specification-authoring problem entirely, scales to project- scale work without us writing build infrastructure, and produces a benchmark whose top-line we trust after the audit. RTL-UNIVERSE as it stands is not the right benchmark forever, but the restoration framing has more headroom than what we had before.

8 Final-Artifact Inventory

The deliverables across the five repositories are summarised below. All five repositories are committed (RTL-UNIVERSE on the codex-experiments-202605 branch of github.com/cs3220-research/rtl-universe; the other four under other_repos/).

- SWE-UNIVERSE-VERILOG: 4 commits, 7-stage pipeline (Python, SQLite-backed, resumable), 401 strict-quality PRs across 3,296 HDL repositories.

- RTL BENCH: 12 commits, 3 Docker images (cocotb 1.x, cocotb 2.x, Verilator), re-implementation of the SWE-bench evaluation harness for HDL, 7 candidate upstream HDL repositories.
- CVD P-HARBOR: 4 of-our commits on top of NVIDIA’s CVD P fork, the Harbor conversion tool, 99 converted Harbor tasks, three frontier-model evaluation results (Opus 81.5%, GPT-5.4 66.3%, Gemini 3.1-Pro 54.3% on 92 tasks), Opus reaching 96.2% on the cleaned subset, and one upstream-bug finding (the 79/92 cocotb harness bug) we patched in our port.
- SPEC2GDS BENCH: 52 commits, 12 task types (8 module-level, 1 SoC, 3 AI accelerators), three weekly research reports totalling roughly 12,000 words plus a 1,042-word phase-1 research summary, 96 result directories spanning four sync generations, three reference designs that pass the full ASIC flow through GDSII on Sky130A.
- RTL-UNIVERSE: 76 commits, the harbor-task-gen skill (with 5 iterations and roughly 20 documented bugfixes in BUGFIXES.md), 11 working restoration tasks (with 7 archived candidates that did not make the cut), the multi-agent launcher (scripts/launcher.py, 616 lines, with launcher_supervisor.sh and test_launcher.py), the sandboxed-agent adapters, the audit verdicts, and the companion benchmark paper.

Acknowledgements. We thank the lowRISC, chipsalliance, NVIDIA, Princeton, PULP, Google Coral, and secworks teams for the upstream projects this benchmark is built on. Repository contributions over this arc are split roughly evenly between the two authors. Our other GitHub handles are Broyojo (David) and spyre (Saketh); the SpiritSeal handle was used during the early SWE-UNIVERSE-VERILOG and RTL BENCH work.

References

- [1] Mingjie Liu, Nathaniel Pinckney, Bruce K Khailany, and Haoxing Ren. Verilog eval: Evaluating large language models for verilog code generation, 2023. URL <https://arxiv.org/abs/2309.07544>.
- [2] Yao Lu, Shang Liu, Qijun Zhang, and Zhiyao Xie. Rtl lm: An open-source benchmark for design rtl generation with large language model, 2023. URL <https://arxiv.org/abs/2308.05345>.
- [3] Shailja Thakur, Baleegh Ahmad, Zhenxing Fan, Hammond Pearce, Benjamin Tan, Ramesh Karri, Brendan Dolan-Gavitt, and Siddharth Garg. Benchmarking large language models for automated verilog rtl code generation, 2022. URL <https://arxiv.org/abs/2212.11140>.
- [4] Shang Liu, Wenji Fang, Yao Lu, Qijun Zhang, Hongce Zhang, and Zhiyao Xie. Rtlcoder: Outperforming gpt-3.5 in design rtl generation with our open-source dataset and lightweight solution, 2025. URL <https://arxiv.org/abs/2312.08617>.
- [5] Shang Liu, Yao Lu, Wenji Fang, Mengming Li, and Zhiyao Xie. Openllm-rtl: Open dataset and benchmark for llm-aided design rtl generation, 2025. URL <https://arxiv.org/abs/2503.15112>.
- [6] Zehua Pei, Hui-Ling Zhen, Mingxuan Yuan, Yu Huang, and Bei Yu. Betterv: Controlled verilog generation with discriminative guidance, 2024. URL <https://arxiv.org/abs/2402.03375>.
- [7] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2024. URL <https://arxiv.org/abs/2310.06770>.
- [8] Mouxiang Chen, Lei Zhang, Yunlong Feng, Xuwu Wang, Wenting Zhao, Ruisheng Cao, Jiaxi Yang, Jiawei Chen, Mingze Li, Zeyao Ma, Hao Ge, Zongmeng Zhang, Zeyu Cui, Dayiheng Liu, Jingren Zhou, Jianling Sun, Junyang Lin, and Binyuan Hui. Swe-universe: Scale real-world verifiable environments to millions, 2026. URL <https://arxiv.org/abs/2602.02361>.
- [9] NVIDIA Research. CVD P: Comprehensive Verilog design problems benchmark. <https://github.com/NVlabs/CVD P-benchmark>, 2024.

- [10] David Andrews and Saketh Reddy. Spec2GDSBench: Implementation and baseline results for end-to-end chip design evaluation. Technical report, Internal weekly research report, March 2026.
- [11] David Andrews and Saketh Reddy. From individual tasks to full SoC: Redesigning spec2GDSBench around linux boot. Technical report, Internal weekly research report, April 2026.